Practica p5 Iker Núñez UO306425

1.Definicion del estado

En mi caso, utilizo una matriz de dos dimensiones, donde las filas representan el número de vehículos y las columnas representan la ocupación de babor. La ocupación de estribor no hace falta representarla en la matriz, ya que la podemos deducir de la siguiente formula: Ocupación Estribor = SumaAcumuladaLongitudes – Ocupación Babor.

2.Relacion de recurrencia

Caso base -> es el estado de la fila 0. La celda (0,0) es siempre **true**, ya que con 0 vehículos cargados tienes una ocupación de cero metros en babor. El resto de las columnas de esta fila se inicializan a **false** ya que no hay vehículos cargados.

Una celda cualquiera de la matriz se pondrá a **true** si cumple una de estas dos condiciones:

-Condición babor:

El vehículo se puede meter en babor. Para esto, debemos estar en una columna que iguale al menos los metros que ocupa el vehículo (no podemos decir que un vehículo de 4 metros ocupa 3 metros en babor). La segunda parte de la condición es que la celda (a,b) sea **true**, donde a es la fila anterior y b es la columna en la que estas menos la longitud del vehículo que metes.

¿Por qué? Porque si dices que ahora la ocupación de babor va a ser la que había mas la longitud del nuevo vehículo, entonces antes la ocupación antes debía ser la de la columna en la que estas menos la long del vehículo (j – longVeh).

La condición de babor en código se vería así:

```
//¿Puedes meter en babor?
if(j >= longVeh && matrizEstado[i - 1][j - longVeh])
```

-Condición estribor:

El vehículo se puede meter en estribor. Esta condición, como la anterior tiene 2 partes. La primera, la celda (i-1,j), es decir, la que esta inmediatamente encima, debe ser **true.** ¿Por qué? Porque si metes un coche en estribor, la ocupación de babor no cambiara, luego miras que babor tenga esos metros ocupados.

La segunda parte de la condición es que el vehículo que quieres meter en estribor coja en este, para ello miras que la longitud de estribor (incluyendo ya este nuevo vehículo) sea menor o igual que la máxima por carril.

La condición de estribor en código se vería así:

```
//Si no, ¿puedo meter en estribor?
else if(matrizEstado[i-1][j] && longEstribor <= longitudCarril)
```

3.Complejidad del algoritmo en función de N (nº vehículos) y L (longitud del carril)

La complejidad de este algoritmo en función de N y L, resulta ser de O(N * L), ya que, para cada vehículo, recorres todas las L columnas.

4.Implementacion

En la carpeta se adjunta la clase Ferry.java